

Липецкий государственный технический университет

Институт компьютерных наук
Кафедра прикладной математики и системного анализа

Отчет по лабораторной работе № 1
по дисциплине «Численные методы»
на тему «Интерполяция функций»

Студент		Сараев А.П.
Группа ПМ-24-1	подпись, дата	фамилия, инициалы
Руководитель		Супрунов И.И.
к.т.н., доцент		
учёная степень, учёное звание	подпись, дата	фамилия, инициалы

1 Задание

Разработать программное обеспечение для построения интерполяционных многочленов Лагранжа и Ньютона на основе произвольно заданной табличной функции. Рассчитать значение функции в заданной пользователем точке x^* на основе построенных интерполяционных многочленов, а так-же при помощи схемы Эйткена.

Предусмотреть вывод интерполяционных многочленов Лагранжа и Ньютона в изначальном и упрощенном виде (по убыванию степеней), а также вывод результатов вычислений разделенных разностей для многочлена Ньютона и промежуточных значений многочленов Лагранжа в схеме Эйткена. Показать значение в точке x^* , полученное по схеме Эйткена и при подстановке в каждый из интерполяционных многочленов.

Сравнить результаты, полученные разными способами.

Организовать обработку исключений:

1. ввод некорректного символа, значения;
2. точка x^* находится вне отрезка интерполяции;
3. узлы не упорядочены по возрастанию.

2 Теоретические сведения

2.1 Интерполяционный многочлен Лагранжа. Схема Эйткена

Рассматриваются многочлены

$$\varphi(x) = \sum_{i=0}^n a_i x^i$$

с базисными функциями

$$1, x, x^2, \dots, x^n.$$

Тогда

$$|\Phi| = \begin{vmatrix} 1 & x_0 & \dots & x_0^n \\ 1 & x_1 & \dots & x_1^n \\ \vdots & \vdots & & \vdots \\ 1 & x_n & \dots & x_n^n \end{vmatrix} = \prod_{i>j} (x_i - x_j).$$

Для выполнения условий

$$\Phi_i(x_j) = \delta_{ij}$$

фундаментальные многочлены имеют вид

$$\Phi_i(x) = c_i(x - x_0) \dots (x - x_{i-1})(x - x_{i+1}) \dots (x - x_n),$$

откуда $\Phi_i(x_j) = 0$ при $i \neq j$. Из условия нормировки

$$\Phi_i(x_i) = c_i(x_i - x_0) \dots (x_i - x_{i-1})(x_i - x_{i+1}) \dots (x_i - x_n) = 1.$$

Следовательно,

$$\Phi_i(x) = \frac{(x-x_0)\dots(x-x_{i-1})(x-x_{i+1})\dots(x-x_n)}{(x_i-x_0)\dots(x_i-x_{i-1})(x_i-x_{i+1})\dots(x_i-x_n)} = \frac{\prod_{\substack{k=0 \\ k \neq i}}^n (x-x_k)}{\prod_{\substack{k=0 \\ k \neq i}}^n (x_i-x_k)}. \quad (1)$$

Это фундаментальные многочлены Лагранжа.

Интерполяционный многочлен Лагранжа:

$$\varphi(x) = L_n(x) = \sum_{i=0}^n f_i \frac{\prod_{\substack{k=0 \\ k \neq i}}^n (x-x_k)}{\prod_{\substack{k=0 \\ k \neq i}}^n (x_i-x_k)}. \quad (2)$$

Узловой многочлен:

$$\omega(x) = (x-x_0)(x-x_1)\dots(x-x_n) = \prod_{k=0}^n (x-x_k).$$

Тогда многочлен Лагранжа можно записать в виде

$$L_n(x) = \sum_{i=0}^n \frac{f_i \omega(x)}{(x-x_i) \omega'(x_i)} = \omega(x) \sum_{i=0}^n \frac{f_i}{(x-x_i) \omega'(x_i)}.$$

При больших n вычисления громоздки. Вычисления значений многочленов Лагранжа различных порядков в заданной точке x удобно проводить рекуррентным образом по интерполяционной схеме Эйткена:

$$L_i(x) = f_i,$$

$$L_{0,1} = \frac{1}{x_1-x_0} \begin{vmatrix} x-x_0 & L_0 \\ x-x_1 & L_1 \end{vmatrix}$$

— интерполяционный многочлен Лагранжа 1-го порядка;

$$L_{1,2} = \frac{1}{x_2-x_1} \begin{vmatrix} x-x_1 & L_1 \\ x-x_2 & L_2 \end{vmatrix}.$$

Далее:

$$L_{0,1,2} = \frac{1}{x_2-x_0} \begin{vmatrix} x-x_0 & L_{0,1} \\ x-x_2 & L_{1,2} \end{vmatrix}$$

— интерполяционные многочлены 2-го порядка;

$$L_{0,1,\dots,i} = \frac{1}{x_i-x_0} \begin{vmatrix} x-x_0 & L_{0,1,\dots,i-1} \\ x-x_i & L_{1,2,\dots,i} \end{vmatrix}$$

— интерполяционные многочлены порядка i .

2.2 Интерполяционный многочлен Ньютона

Разделённые разности 1-го порядка:

$$f(x_0, x_1) = \frac{f_1 - f_0}{x_1 - x_0}, \quad f(x_1, x_2) = \frac{f_2 - f_1}{x_2 - x_1}, \quad f(x_{n-1}, x_n) = \frac{f_n - f_{n-1}}{x_n - x_{n-1}}.$$

Разделённые разности 2-го порядка:

$$f(x_0, x_1, x_2) = \frac{f(x_1, x_2) - f(x_0, x_1)}{x_2 - x_0},$$
$$f(x_{n-2}, x_{n-1}, x_n) = \frac{f(x_{n-1}, x_n) - f(x_{n-2}, x_{n-1})}{x_n - x_{n-2}}.$$

Разделённые разности $(k+1)$ -го порядка:

$$f(x_{i-1}, x_i, \dots, x_{i+k}) = \frac{f(x_i, x_{i+1}, \dots, x_{i+k}) - f(x_{i-1}, x_i, \dots, x_{i+k-1})}{x_{i+k} - x_{i-1}}.$$

Заметим, что

$$f(x_i, x_{i+1}, \dots, x_{i+k}) = \sum_{j=0}^k \frac{f(x_{i+j})}{(x_{i+j} - x_i) \dots (x_{i+j} - x_{i+j-1})(x_{i+j} - x_{i+j+1}) \dots (x_{i+j} - x_{i+k})}.$$

Пусть x — произвольная фиксированная точка. Тогда

$$f(x, x_0) = \frac{f(x_0) - f(x)}{x_0 - x},$$

$$f(x) = f(x_0) + f(x, x_0)(x - x_0).$$

Для разделённой разности 2-го порядка:

$$f(x, x_0, x_1) = \frac{f(x_0, x_1) - f(x, x_0)}{x_1 - x},$$

$$f(x, x_0) = f(x_0, x_1) + f(x, x_0, x_1)(x - x_1).$$

Интерполяционная формула Ньютона для неравноотстоящих узлов:

$$f(x) \approx P_n(x) = f(x_0) + f(x_0, x_1)(x - x_0) + f(x_0, x_1, x_2)(x - x_0)(x - x_1) + \dots$$
$$\dots + f(x_0, x_1, \dots, x_n)(x - x_0)(x - x_1) \dots (x - x_{n-1}).$$

Остаточный член:

$$R_n(x) = f(x) - P_n(x) = f(x, x_0, x_1, \dots, x_n)\omega(x).$$

Вычисления разделённых разностей удобно представлять в виде таблицы.

3 Листинг программы

3.1 Реализация метода Лагранжа

```
#include <vector>

class Lagrange {
    std::vector<double>polynom;
    std::vector<double>x,y;
    int n;
public:
    Lagrange(std::vector<double>n_x, std::vector<double>n_y) : x(n_x), y(n_y), n(x
.size()) {
        polynom.resize(n, 0);
        buildPolynom();
    }

    void buildPolynom() {
        for (int i = 0; i < n; i++) {
            if (y[i] == 0) continue;

            double mul = y[i];
            for (int j = 0; j < n; j++) {
                if (j != i) mul /= (x[i] - x[j]);
            }

            std::vector<double>cur(2);
            cur[1] = 1;
            cur[0] = -(i == 0 ? x[1] : x[0]);
            for (int j = 0; j < n; j++) {
                if (i == j) continue;
                if (i == 0 && j == 1) continue;
                if (i >= 1 && j == 0) continue;
                std::vector<double>next(cur.size() + 1, 0);
                next[0] = -1 * cur[0] * x[j];
                for (int k = 1; k < next.size() - 1; k++) {
                    next[k] = cur[k - 1] - x[j] * cur[k];
                }
                next[next.size() - 1] = cur[cur.size() - 1];
                cur = next;
            }
            for (int j = 0; j < n; j++) {
                polynom[j] += cur[j] * mul;
            }
        }
    }

    std::vector<double> getPolynom() {
        return polynom;
    }

    double getValueInPoint(double find_x) {
        double value = 0;
```

```

        for (int i = n - 1; i >= 0; i--) {
            value = value * find_x + polynom[i];
        }
        return value;
    }
};

```

3.2 Реализация метода Ньютона

```

#include <vector>
#include <iostream>

class Newton {
    std::vector<double>polynom;
    std::vector<double>x,y;
    int n;
    std::vector<std::vector<double>>>rr;

public:
    Newton(std::vector<double>n_x, std::vector<double>n_y) : x(n_x), y(n_y), n(x.
size()) {
        polynom.resize(n, 0);
        buildRR();
        buildPolynom();
    }

    void buildRR() {
        rr.resize(n);
        for (int i = 0; i < n; i++) rr[0].push_back(y[i]);
        for (int i = 1; i < n; i++) {
            for (int j = 0; j < n - i; j++) {
                rr[i].push_back((rr[i - 1][j + 1] - rr[i - 1][j]) / (x[i + j] - x[
j]));
            }
        }
    }

    void buildPolynom() {
        polynom[0] = y[0];

        for (int i = 1; i < n; i++) {
            std::vector<double>cur(2);
            cur[0] = -x[0];
            cur[1] = 1;
            for (int j = 1; j < i; j++) {
                std::vector<double>next(cur.size() + 1, 0);
                next[0] = -1 * cur[0] * x[j];
                for (int k = 1; k < next.size() - 1; k++) {
                    next[k] = cur[k - 1] - x[j] * cur[k];
                }
                next[next.size() - 1] = cur[cur.size() - 1];
                cur = next;
            }
        }
    }
};

```

```

        }

        for (int j = 0; j < cur.size(); j++) {
            polynom[j] += cur[j] * rr[i][0];
        }
    }

std::vector<double> getPolynom() {
    return polynom;
}

double getValueInPoint(double find_x) {
    double value = 0;
    for (int i = n - 1; i >= 0; i--) {
        value = value * find_x + polynom[i];
    }
    return value;
}

std::vector<std::vector<double>> getRR() {
    return rr;
}
};

```

3.3 Реализация метода Эйткена

```

#include <vector>

class Aitken {
    std::vector<double>x,y;
    int n;

public:
    Aitken(std::vector<double>n_x, std::vector<double>n_y) : x(n_x), y(n_y), n(x.size()) {}

    double getValueInPoint(double find_x) {
        std::vector<double>polynoms = y;

        for (int i = 1; i < n; i++) {
            std::vector<double>next(n - i);
            for (int j = 0; j < n - i; j++) {
                next[j] = (polynoms[j + 1] * (find_x - x[j])
                    - polynoms[j] * (find_x - x[i + j]))
                    / (x[i + j] - x[j]);
            }
            polynoms = next;
        }
        return polynoms[0];
    }
}

```

```

std::vector<std::vector<double>> buildTable(double findX) {
    const int n = static_cast<int>(x.size());
    std::vector<std::vector<double>> table(n);

    table[0] = y;

    for (int k = 1; k < n; ++k) {
        table[k].resize(n - k);

        for (int j = 0; j < n - k; ++j) {
            table[k][j] =
                (table[k - 1][j + 1] * (findX - x[j])
                 - table[k - 1][j] * (findX - x[j + k]))
                / (x[j + k] - x[j]);
        }
    }

    return table;
}
};

```

3.4 Реализация вывода полинома

```

string polynomialFromCoefficients(const vector<double>& c) {
    string result;
    const double EPS = 1e-6;

    for (int i = static_cast<int>(c.size()) - 1; i >= 0; --i) {
        if (std::abs(c[i]) < EPS) continue;

        double a = c[i];
        bool negative = a < 0;
        double absA = std::abs(a);

        if (result.empty()) {
            if (negative) result += "-";
        } else {
            result += negative ? " - " : " + ";
        }

        if (i == 0) {
            result += num(absA);
        } else {
            if (std::abs(absA - 1.0) > EPS) {
                result += num(absA) + "*";
            }
            result += variablePower(i);
        }
    }

    return result.empty() ? "0" : result;
}

```

4 Пример работы

```
=====
Интерполяция Лагранжа, Ньютона и схема Эйткена
=====

Введите количество узлов интерполяции (n >= 2): 4

Введите узлы таблицы в порядке возрастания.
x[0] = -1
y[0] = 4
x[1] = 0
y[1] = 2
x[2] = 1
y[2] = 0
x[3] = 2
y[3] = 1

Введите точку x* = 1.5
```

Рис. 1: Ввод

```
===== МНОГОЧЛЕН ЛАГРАНЖА =====

Исходный вид:
L(x) = (4)*((x - (0))*(x - (1))*(x - (2)))/(-6) + (2)*((x - (-1))*(x - (1))*(x - (2)))/(2)
+ (0)*((x - (-1))*(x - (0))*(x - (2)))/(-2) + (1)*((x - (-1))*(x - (0))*(x - (1)))/(6)

Упрощённый вид (по убыванию степеней):
L(x) = 0.5*x^3 - 2.5*x + 2

===== МНОГОЧЛЕН НЬЮТОНА =====

Исходный вид:
N(x) = 4 + (-2)*(x - (-1)) + (0)*(x - (-1))(x - (0)) + (0.5)*(x - (-1))(x - (0))(x - (1))

Упрощённый вид (по убыванию степеней):
N(x) = 0.5*x^3 - 2.5*x + 2

Разделённые разности для многочлена Ньютона:
Порядок 0: 4      2      0      1
Порядок 1: -2     -2     1
Порядок 2: 0      1.5
Порядок 3: 0.5

===== СХЕМА ЭЙТКЕНА =====

Промежуточные значения схемы Эйткена:
P[0]: P(0,0) = 4      P(1,1) = 2      P(2,2) = 0      P(3,3) = 1
P[1]: P(0,1) = -1     P(1,2) = -1     P(2,3) = 0.5
P[2]: P(0,2) = -1     P(1,3) = 0.125
P[3]: P(0,3) = -0.0625

Значение по схеме Эйткена:
P(x*) = -0.0625
```

Рис. 2: Вывод трёх методов

```

===== РЕЗУЛЬТАТЫ =====
x* = 1.5

Лагранж : -0.0625
Ньютон  : -0.0625
Эйткен  : -0.0625

Абсолютные разности:
|Лагранж - Ньютон| = 0
|Лагранж - Эйткен| = 0
|Ньютон - Эйткен| = 0

```

Рис. 3: Результат работы программы

```

Введите количество узлов интерполяции (n >= 2): grfgwe
Ошибка: введите целое число от 2 до 100.
Введите количество узлов интерполяции (n >= 2): 0 1 2
Ошибка: введите целое число от 2 до 100.
Введите количество узлов интерполяции (n >= 2): 2.4
Ошибка: введите целое число от 2 до 100.
Введите количество узлов интерполяции (n >= 2): -1
Ошибка: введите целое число от 2 до 100.

```

Рис. 4: Обработка ошибок при вводе кол-ва точек

```

Введите узлы таблицы в порядке возрастания.
x[0] = f
Ошибка: введите корректное числовое значение.
x[0] = few
Ошибка: введите корректное числовое значение.
x[0] = 2 3

```

Рис. 5: Обработка ошибок при вводе координат точек

```

x[0] = -1
y[0] = 4
x[1] = 0
y[1] = 2
x[2] = 1
y[2] = 0
x[3] = 2
y[3] = 1

Введите точку x* = 5

Ошибка диапазона: Точка x* находится вне отрезка интерполяции [x0, xn-1].

```

Рис. 6: Обработка ошибок при вводе x^* вне отрезка интерполяции

```

Введите количество узлов интерполяции (n >= 2): 5

Введите узлы таблицы в порядке возрастания.
x[0] = -0.12
y[0] = 1.321
x[1] = -0.005
y[1] = -2.481
x[2] = 0.013
y[2] = -1.3333
x[3] = 0.0497
y[3] = 0.589
x[4] = 0.1657
y[4] = 0.7965

Введите точку x* = 0.0576

```

Рис. 7: Тестовый пример

```

Упрощённый вид (по убыванию степеней):
N(x) = 16780.6*x^4 - 4470.61*x^3 - 0.419163*x^2 + 64.3151*x - 2.15998

Разделённые разности для многочлена Ньютона:
Порядок 0: 1.321      -2.481      -1.3333      0.589      0.7965
Порядок 1: -33.0609    63.7611    52.3787    1.78879
Порядок 2: 727.985    -208.087   -331.303
Порядок 3: -5516.04   -721.827
Порядок 4: 16780.6

===== СХЕМА ЭЙТКЕНА =====

Промежуточные значения схемы Эйткена:
P[0]: P(0,0) = 1.321      P(1,1) = -2.481      P(2,2) = -1.3333
      P(3,3) = 0.589      P(4,4) = 0.7965
P[1]: P(0,1) = -4.55061    P(1,2) = 1.51045    P(2,3) = 1.00
279      P(3,4) = 0.603131
P[2]: P(0,2) = 3.54295     P(1,3) = 0.929475    P(2,4) = 0.88
6061
P[3]: P(0,3) = 0.80781     P(1,4) = 0.913554
P[4]: P(0,4) = 0.873544

Значение по схеме Эйткена:
P(x*) = 0.873544

```

Рис. 8: Результат своей программы

Полином Ньютона после упрощений

$$16780.58x^4 - 4470.61x^3 - 0.42x^2 + 64.32x - 2.16$$

Интерполированные значения

x

0.06

y

0.8735436843704688

Рис. 9: Результат сторонней программы

5 Вывод

В ходе лабораторной работы была разработана программа для интерполяции таблично заданной функции.

В программе реализовано использование трёх подходов:

- интерполяционного многочлена Лагранжа;
- интерполяционного многочлена Ньютона;
- схемы Эйткена.

Для метода Ньютона была сформирована таблица разделённых разностей, а для схемы Эйткена выведены промежуточные результаты рекуррентных вычислений.

Кроме вычислительной части, в программе реализована обработка некорректных входных данных: проверяется корректность числового ввода, строгая возрастаемость узлов интерполяции и нахождение точки x^* внутри интервала интерполяции.